SQL Lab-22

Implement Advance Common Table Expressions (CTE) for Query Simplification

From the table CUSTOMER perform the following queries:

Part – A:

1. Display top 3 highest amount orders.

```
WITH CTE AS (
   SELECT c.*,
        RANK() OVER (ORDER BY amount DESC) AS rnk
   FROM customer c
)
SELECT *
FROM CTE
WHERE rnk <= 3;
```

2.Display second highest order amount.

```
WITH CTE AS (
   SELECT amount,
        DENSE_RANK() OVER (ORDER BY amount DESC) AS rnk
   FROM customer
)
SELECT DISTINCT amount AS second_highest_amount
FROM CTE
WHERE rnk = 2;
```

3.Display customers whose order amount is greater than category average amount.

```
WITH CTE AS (
   SELECT category,
        AVG(amount) AS avg_amount
   FROM customer
   GROUP BY category
)
SELECT c.*
FROM customer c
JOIN CTE  a
ON c.category = a.category
WHERE c.amount > a.avg_amount;
```

4.Display categories having average amount greater than 30000.

```
WITH CTE AS (
   SELECT category,
        AVG(amount) AS avg_amount
   FROM customer
   GROUP BY category
)
SELECT *
FROM CTE
WHERE avg_amount > 30000;
```

5.Display highest amount order from each category.

```
WITH CTE AS (
   SELECT c.*,
        RANK() OVER (
           PARTITION BY category
           ORDER BY amount DESC
```

```
        ) AS rnk
    FROM customer c
)
SELECT *
FROM CTE
WHERE rnk = 1;
```

6.Display lowest amount order from each category.

```
WITH CTE AS (
SELECT c.*,
        RANK() OVER (
            PARTITION BY category
            ORDER BY amount
        ) AS rnk
    FROM customer c
)
SELECT *
FROM CTE
WHERE rnk = 1;
```

7.Display categories having more than 3 orders.

```
WITH CTE AS (
    SELECT category,
        COUNT(*) AS order_count
    FROM customer
    GROUP BY category
)
SELECT *
FROM CTE
WHERE order_count > 3;
```

8.Display city-wise total order amount.

```
WITH CTE AS (
   SELECT city,
        SUM(amount) AS total_amount
   FROM customer
   GROUP BY city
)
SELECT *
FROM CTE
ORDER BY city;
```

9.Display category having highest average order amount.

```
WITH category_avg AS (
   SELECT category,
        AVG(amount) AS avg_amount
   FROM customer
   GROUP BY category
),
ranked_category AS (
SELECT category,
avg_amount,
        RANK() OVER (ORDER BY avg_amount DESC) AS rnk
   FROM category_avg
)
SELECT category, avg_amount
FROM ranked_category
WHERE rnk = 1;
```

10.Display cumulative order amount in ascending order of amount.

```
WITH CTE AS (
   SELECT orderid, cname, amount
   FROM customer
)
SELECT orderid,
cname,       amount,
      SUM(amount) OVER (ORDER BY amount) AS cumulative_amount
FROM CTE
ORDER BY amount;
```

Part – B:

11.Display category-wise top 2 highest amount orders.

```
WITH CTE AS (
  SELECT c.*,
      ROW_NUMBER() OVER (
        PARTITION BY category
        ORDER BY amount DESC
      ) AS rn
  FROM customer c
)
SELECT *
FROM CTE
WHERE rn <= 2;
```

12.Display customers whose amount is closest to category average amount.

```
WITH category_avg AS (
  SELECT category,
      AVG(amount) AS avg_amount
  FROM customer
  GROUP BY category
),
customer_difference AS (
  SELECT c.*,
      a.avg_amount,
      ABS(c.amount - a.avg_amount) AS difference
  FROM customer c
  JOIN category_avg a
  ON c.category = a.category
),
ranked_customers AS (
  SELECT *,
```

```
        RANK() OVER (
          PARTITION BY category
          ORDER BY difference
        ) AS rnk
    FROM customer_difference
)
SELECT *
FROM ranked_customers
WHERE rnk = 1;
```

OR

```
WITH CTE1 AS (
    SELECT c.*,
        AVG(amount) OVER (PARTITION BY category) AS avg_amount
    FROM customer c
),
CTE2 AS (
    SELECT CTE1.*,
        ABS(amount - avg_amount) AS difference
    FROM CTE1
)
SELECT *
FROM CTE2
WHERE difference = (
    SELECT MIN(difference)
    FROM CTE2 c
    WHERE c.category = CTE2.category
);
```

13.Display previous, current and next order amount together.

```
WITH CTE AS (
   SELECT orderid,
cname,          amount,
        LAG(amount) OVER (ORDER BY orderid) AS previous_amount,
        LEAD(amount) OVER (ORDER BY orderid) AS next_amount
   FROM customer
)
SELECT orderid,       cname,
previous_amount,       amount
AS current_amount,
next_amount
FROM CTE
ORDER BY orderid;
```

14.Display customers whose amount is greater than previous customer's amount.

```
WITH CTE AS (
   SELECT c.*,
        LAG(amount) OVER (ORDER BY orderid) AS previous_amount
   FROM customer c
)
SELECT *
FROM CTE
WHERE amount > previous_amount;
```

15.Display customers whose rank and dense rank are different.

```
WITH CTE AS (
   SELECT c.*,
        RANK() OVER (ORDER BY amount DESC) AS rnk,
        DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rnk
   FROM customer c
)
SELECT *
FROM CTE
WHERE rnk <> dense_rnk;
```

Part – C:

16.Display orders whose amount is neither highest nor lowest in their category.

```
WITH CTE AS (
  SELECT c.*,
      MAX(amount) OVER (PARTITION BY category) AS highest_amount,
      MIN(amount) OVER (PARTITION BY category) AS lowest_amount
  FROM customer c
)
SELECT *
FROM CTE
WHERE amount > lowest_amount
AND amount < highest_amount
```

17.Display category-wise difference between highest and lowest amount.

```
WITH CTE AS (
  SELECT category,
      MAX(amount) AS highest_amount,
      MIN(amount) AS lowest_amount
  FROM customer
  GROUP BY category
)
SELECT category,      highest_amount,
lowest_amount,      highest_amount -
lowest_amount AS difference
FROM CTE
```

18.Display customers whose amount is greater than all FURNITURE category orders.

```
WITH CTE AS (
   SELECT MAX(amount) AS max_furniture_amount
   FROM customer
   WHERE category = 'FURNITURE'
)
SELECT c.*
FROM customer c, CTE
WHERE c.amount > CTE.max_furniture_amount
```

19.Display categories where all orders are above 10000.

```
WITH CTE AS (
   SELECT category,
        MIN(amount) AS minimum_amount
   FROM customer
   GROUP BY category
)
SELECT category
FROM CTE
WHERE minimum_amount > 10000
```

20.Display customers whose amount difference from category topper is minimum.

```
WITH category_topper AS (
   SELECT category,
        MAX(amount) AS topper_amount
   FROM customer
   GROUP BY category
),
customer_difference AS (
   SELECT c.*,
```

```
        t.topper_amount,
        t.topper_amount - c.amount AS difference
    FROM customer c
    JOIN category_topper t
     ON c.category = t.category
),
ranked_customers AS (
    SELECT *,
        RANK() OVER (
            PARTITION BY category
            ORDER BY difference
        ) AS rnk
    FROM customer_difference
)
SELECT *
FROM ranked_customers
WHERE rnk = 1
```